网络：IP/Socket/HTTP

古金宇

上海交通大学并行与分布式系统研究所

https://ipads.se.sjtu.edu.cn

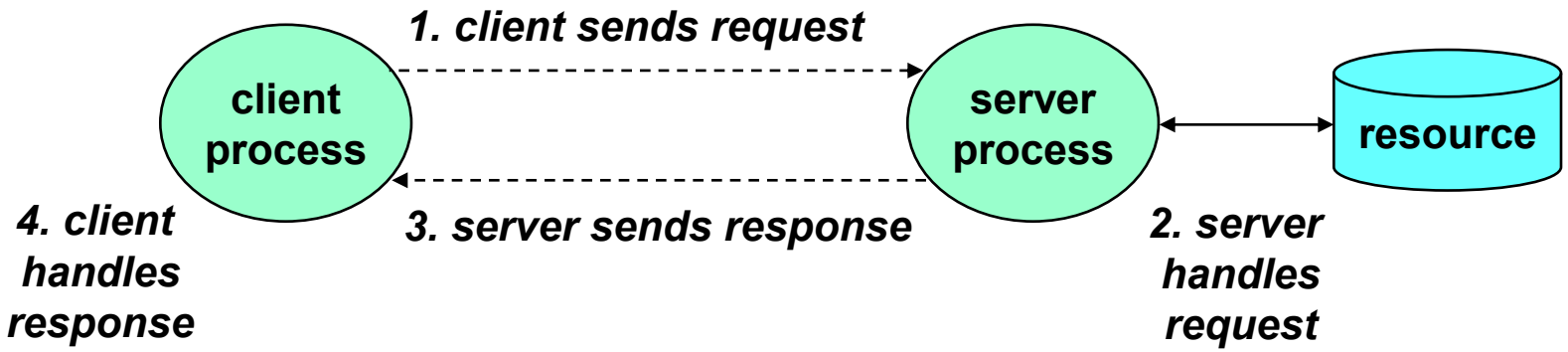
Outline

- Client-Server Programming Model
- Networks and Global IP Internet
- Sockets Interface
- Web server & HTTP Protocol
- Suggested Reading:
  - 11.1~11.6

A client-server transaction

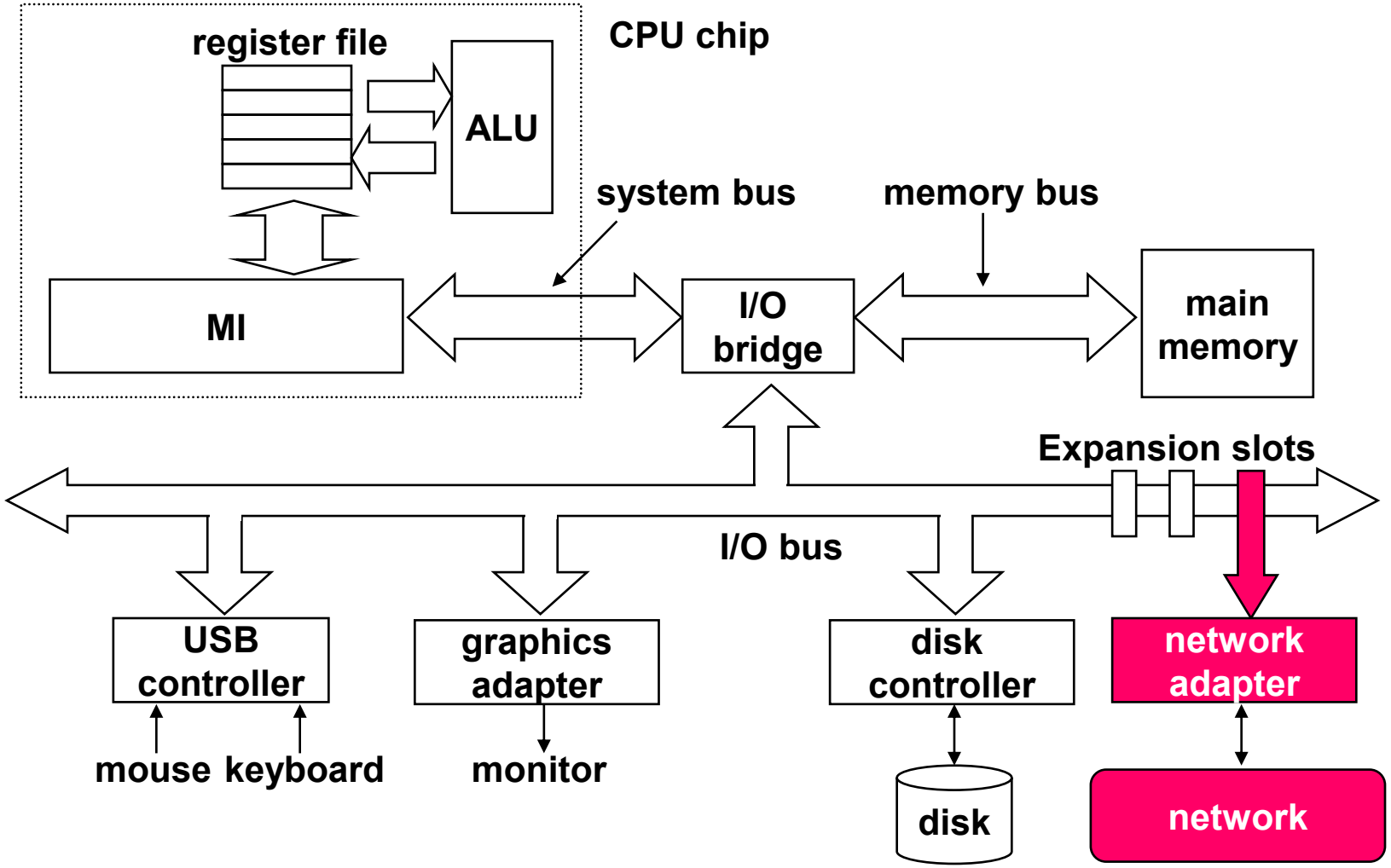
- Most network applications are based on the client-server model
  - A server process and one or more client processes
  - Server manages some resource
  - Server provides service by manipulating resource for clients
  - Server activated by request from client

A client-server transaction



Note: clients and servers are processes running on hosts (can be the same or different hosts).

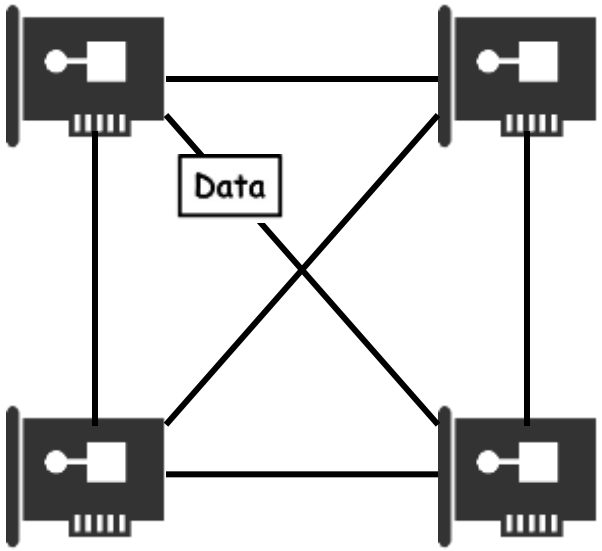
Hardware organization of a network host



Computer networks

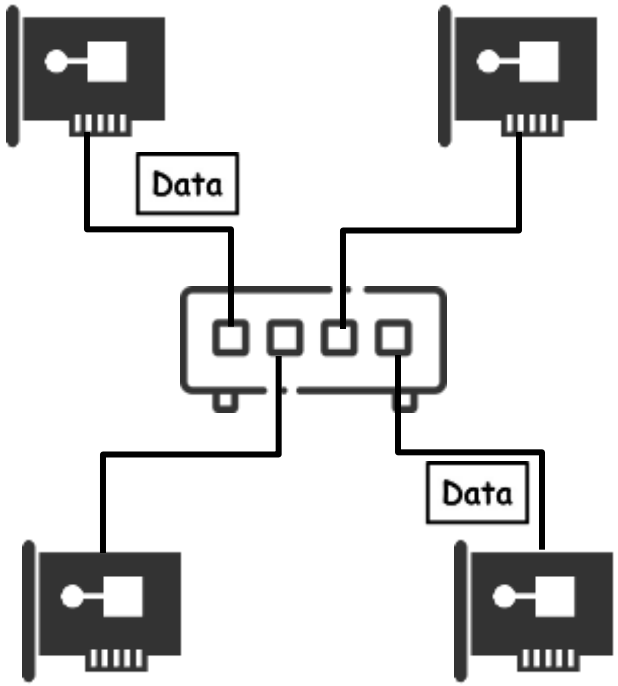
- A network is a hierarchical system of devices (e.g., routers, switches) and communication links (e.g., cables, wireless connections) organized by geographical proximity
- At the lowest level is a LAN (Local Area Network) that spans a building or campus
- The most popular LAN technology is Ethernet

Link Different Hosts in a Small Scale



Direct link them together

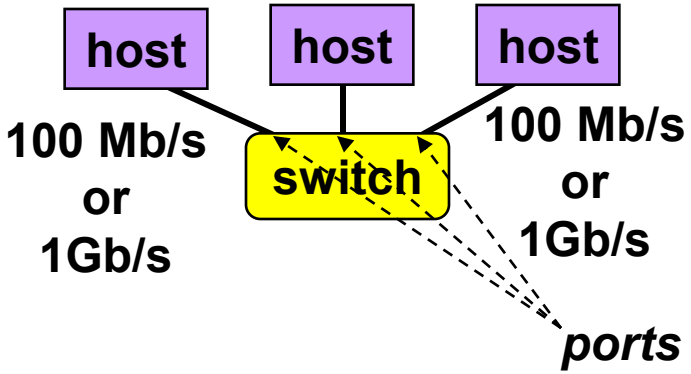
Q: Any issue?



Link to one device: reduce the wires and ports

Ethernet Segment

- Ethernet segment consists of a collection of hosts connected by twisted-pair cables to switches
- Spans room or floor in a building

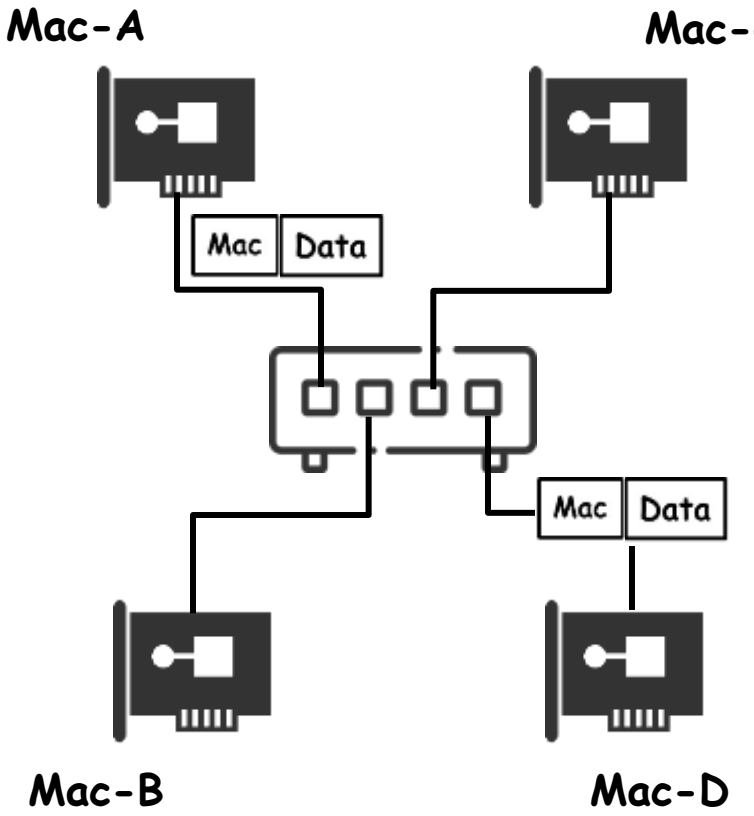


Q: How to differentiate senders and receivers?

Mac Address: Who are the sender & receiver

- Operation
  - Each Ethernet adapter has a unique 48-bit MAC address
    - E.g., `00:16:ea:e3:54:e6`
  - Hosts send bits to any other host in chunks called frames
  - Each frame includes
    - Some fixed number of header bits that identify
      - the source and destination of the frame
      - the frame length
    - Followed by a payload of data bits

Link Different Hosts in a Small Scale



Mac Header

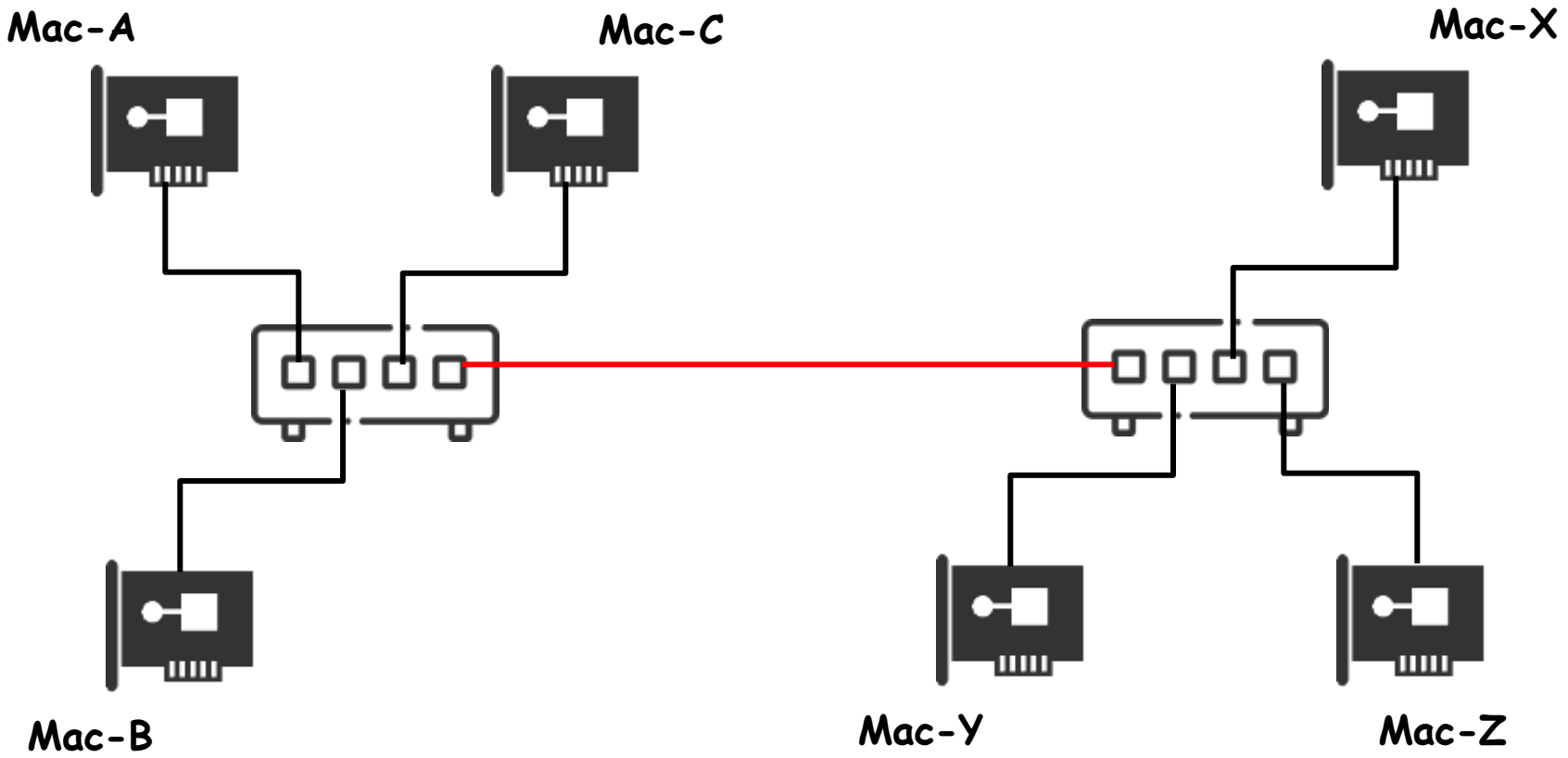
src & dst mac addr	Data

Switch

Port	Mac Addr
1	Mac-A
2	Mac-B
3	Mac-C
4	Mac-D

Q: How can switch construct this Mac Table?

Link Different Hosts in a Large Scale



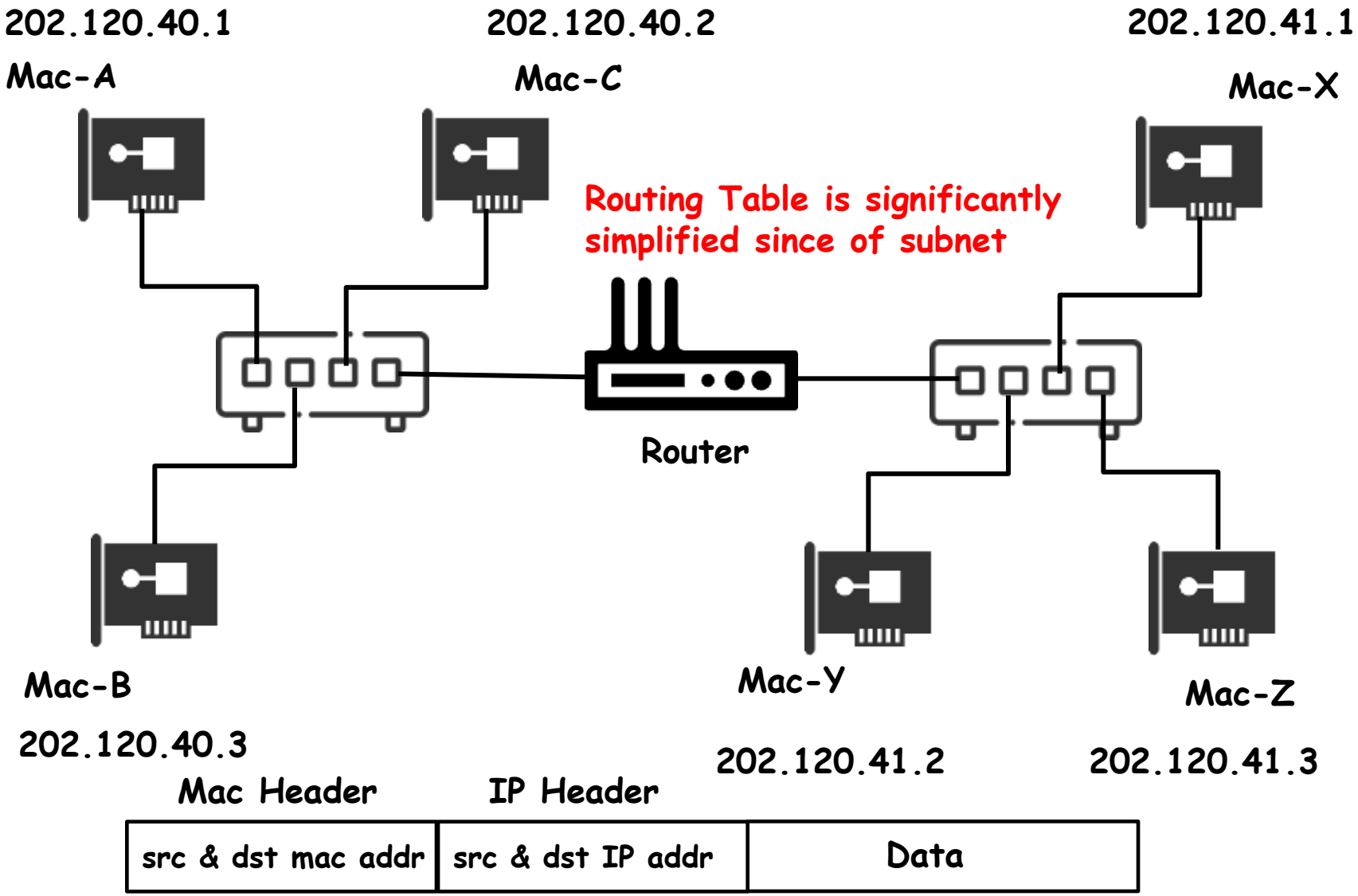
Q: Is this feasible?

Maybe OK for tens or hundreds of hosts.
But it's hart to scale to a large amount, e.g., millions of

Sub-network

- Recall the addresses we used for express delivery
  - Country, province, city, district, street, road, building, room
  - Which is hierarchical
- Subnet
  - The hosts within one subnet have the same address prefix
  - Mac Address does not work as they belong to Network Cards (NICs) decided at manufacture time
  - So, a new address is needed

IP Address and Routing



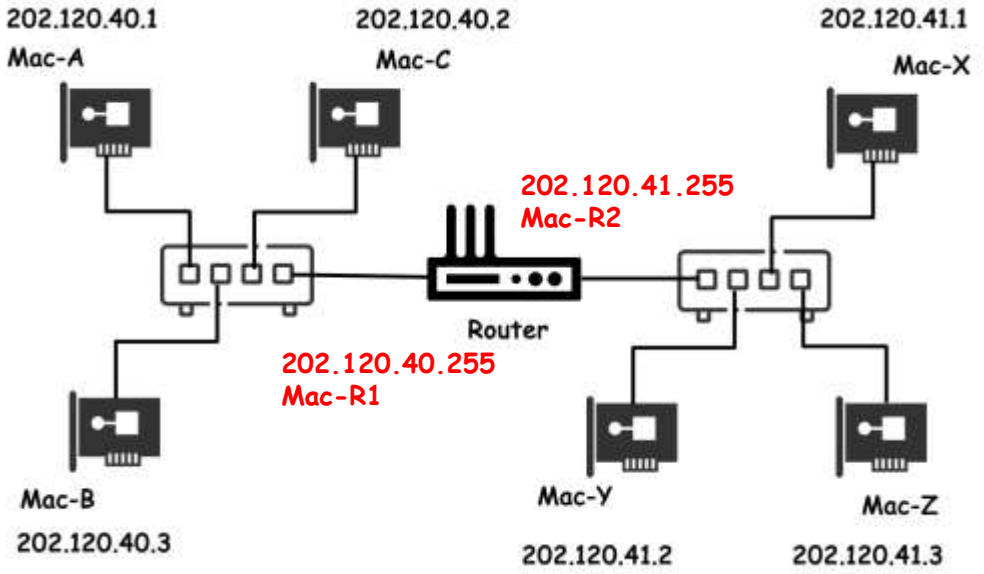
How to Send Data after using IP Addresses

Q1. How does A send data to C?
- Through the switch

Q2. How does A send data to X?
- Through the router

Q3. How does A distinguish C and X?
- Subnet mask (e.g., 255.255.255.0)

Q4. How does A find the router?
- Gateway (e.g., 202.120.40.255)



Recall "子网掩码" and "网关" in the network configuration.

Let us Have a Deeper Look at the Headers

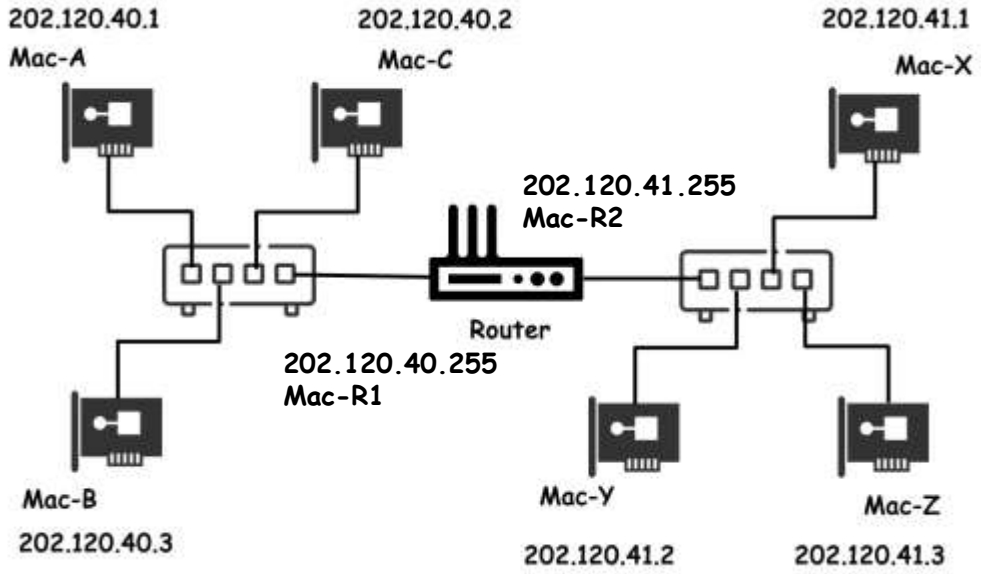
- From A to C

Mac Header	IP Header
Src: Mac-A Dst: Mac-C	Src: 202.120.40.1 Dst: 202.120.40.2

- From A to X

Mac Header	IP Header
Src: Mac-A Dst: Mac-R1	Src: 202.120.40.1 Dst: 202.120.41.1

Mac Header	IP Header
Src: Mac-R2 Dst: Mac-X	Src: 202.120.40.1 Dst: 202.120.41.1



Let us Have a Deeper Look at the Headers

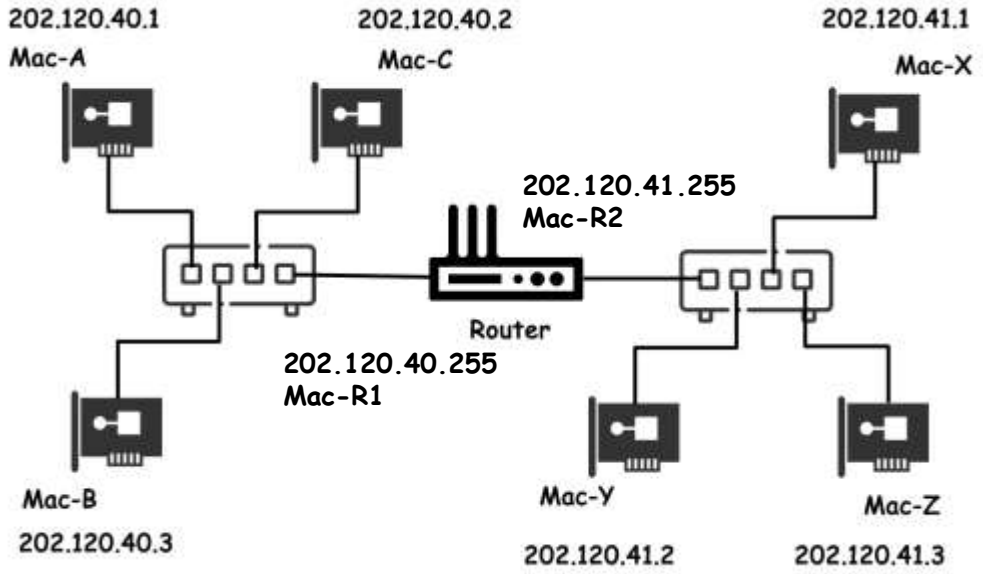
- From A to C

Mac Header	IP Header
Src: Mac-A Dst: Mac-C	Src: 202.120.40.1 Dst: 202.120.40.2

- From A to X

Mac Header	IP Header
Src: Mac-A Dst: Mac-R1	Src: 202.120.40.1 Dst: 202.120.41.1

Mac Header	IP Header
Src: Mac-R2 Dst: Mac-X	Src: 202.120.40.1 Dst: 202.120.41.1

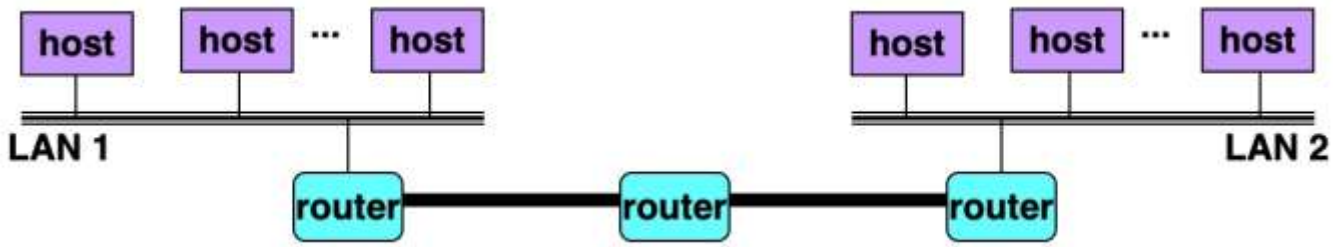


Q: How to know the MAC addr?
The application only states dst IP addr.

ARP: Address Resolution Protocol

Computer networks

- A network is a hierarchical system of devices and wires organized by geographical proximity
- At a higher level in the hierarchy
  - Multiple incompatible LANs can be physically connected by specialized computers called routers
  - The connected networks are called Internet



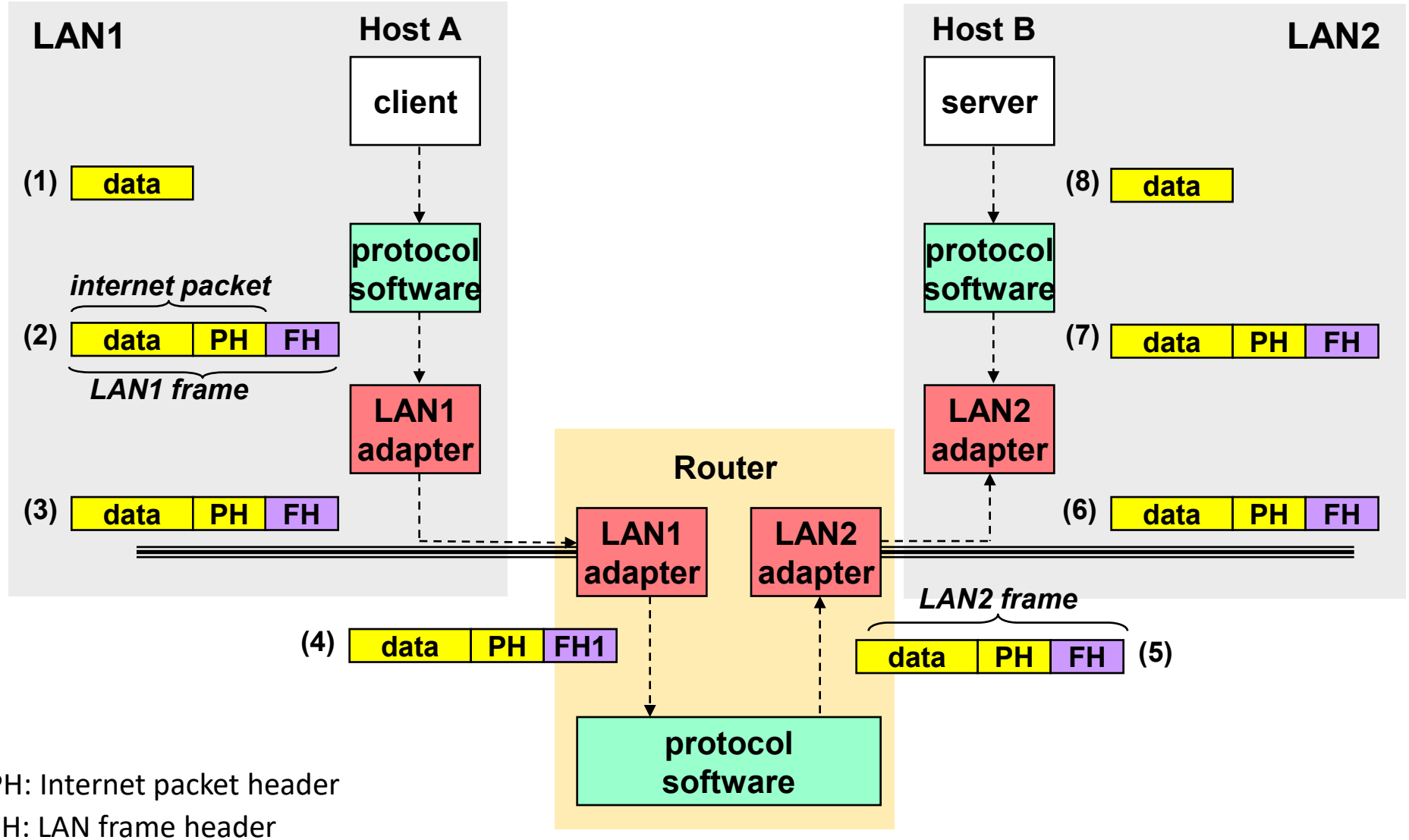
What does an internet protocol (IP) do?

- Provides a naming scheme
  - The internet protocol defines a uniform format for host addresses
  - Each host (and router) is assigned at least one of these internet addresses that uniquely identifies it

What does an internet protocol do?

- Provide a delivery mechanism
  - The internet protocol defines a standard transfer unit (packet)
  - Packet consists of header and payload
    - header: contains info such as packet size, source and destination addresses
    - payload: contains data bits sent from source host

Transferring data over an internet



PH: Internet packet header

FH: LAN frame header

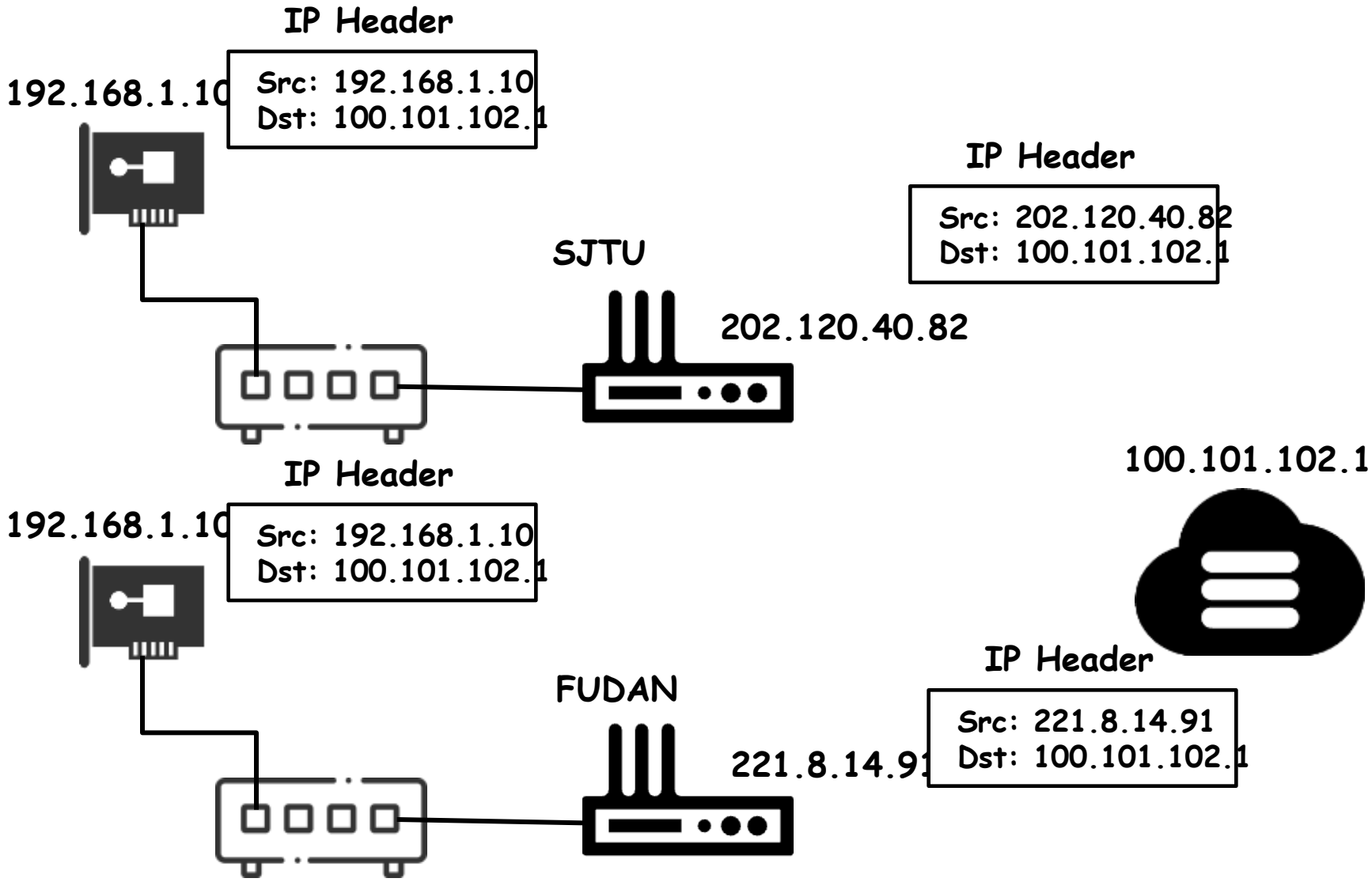
IP within the LAN

- Q: Why 32-bit IP still works for now?

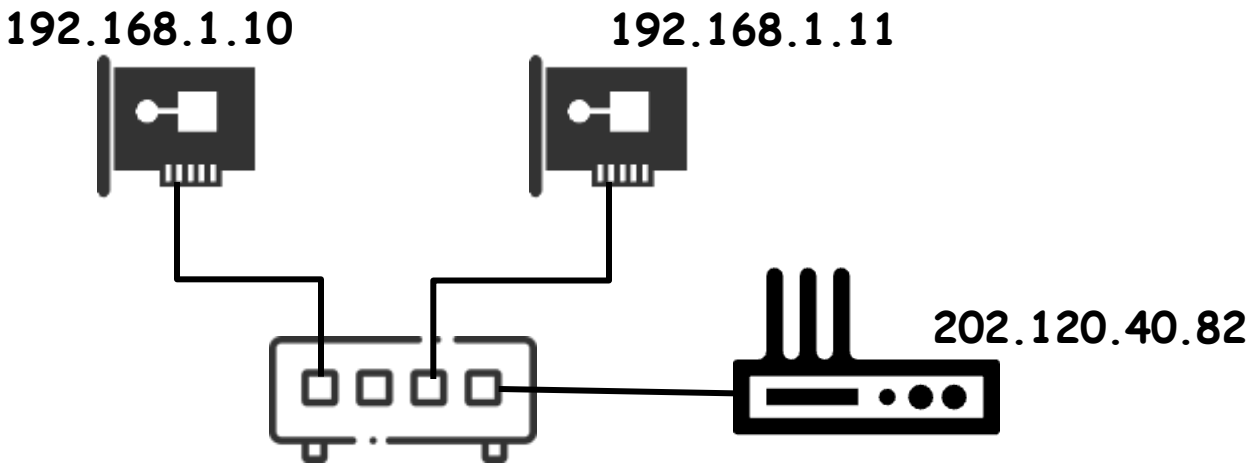
- One PC in SJTU and one PC in FUDAN
  - Have the same IP address: 192.168.1.10
  - They are not the global IP!

- NAT (Network Address Translation)
  - SNAT (S: Src)
  - DNAT (D: Dst)

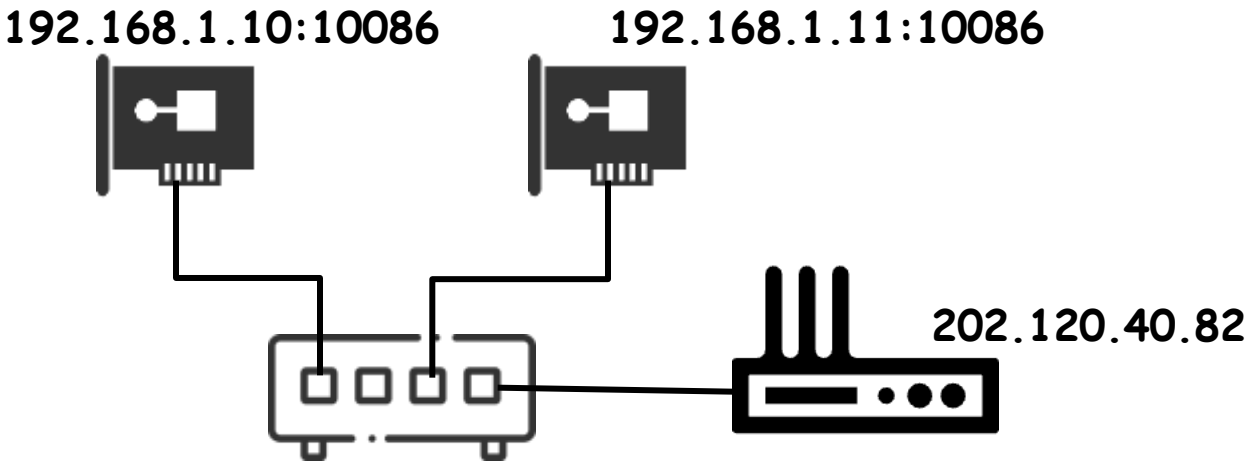
SNAT


IP Header
192.168.1.10
Src: 192.168.1.10
Dst: 100.101.102.1
IP Header
Src: 202.120.40.82
Dst: 100.101.102.1
SJTU
202.120.40.82
100.101.102.1
IP Header
192.168.1.10
Src: 192.168.1.10
Dst: 100.101.102.1
IP Header
Src: 221.8.14.91
Dst: 100.101.102.1
FUDAN
221.8.14.91

SNAT



SNAT



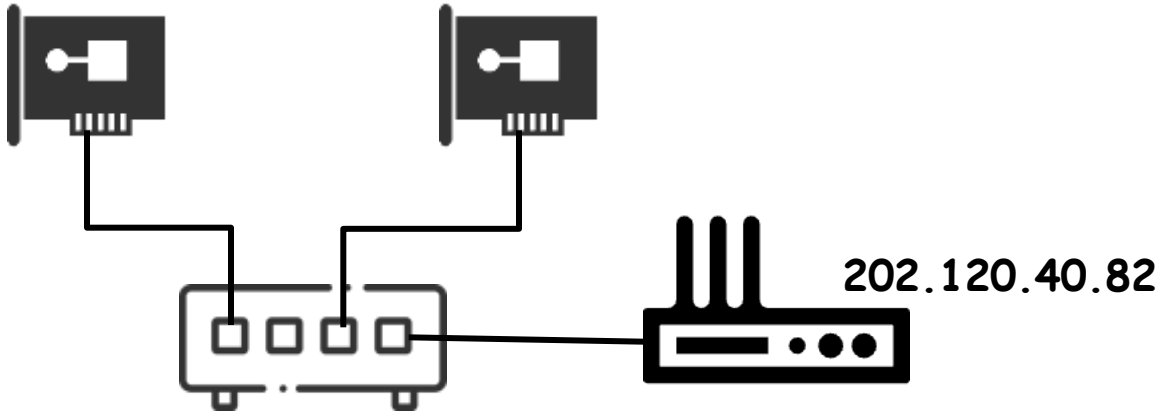
NAT Table

Local	Global
192.168.10:10086	202.120.40.82:233
192.168.11:10086	202.120.40.82:234
…	…

DNAT

Web: Book Store

192.168.1.10:10086

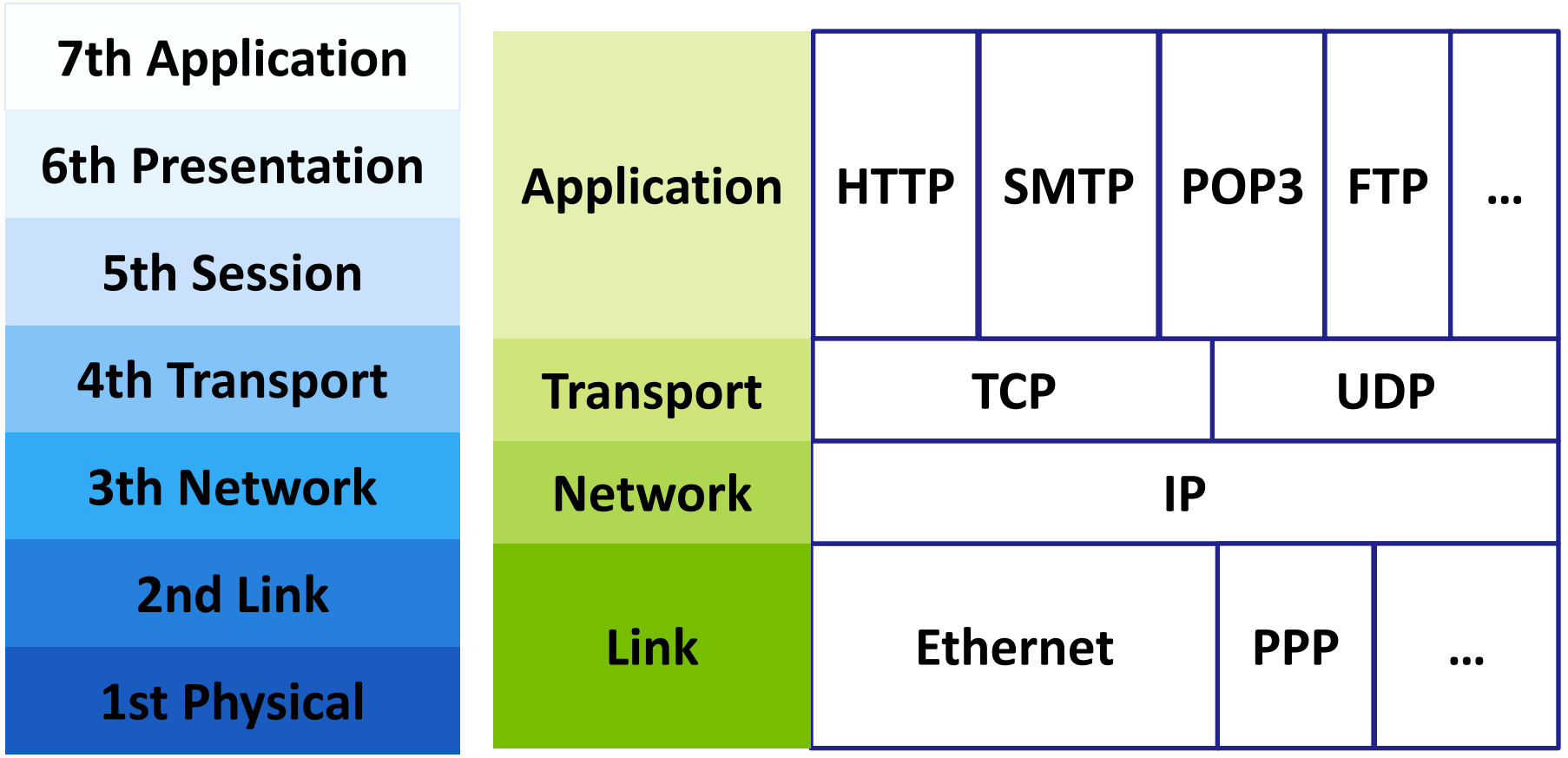


DNAT Table

Port	Local
8080	192.168.1.10:10086
...	...

FROM THE VIEW OF PROGRAMMER

OSI, TCP/IP & Protocol Stack

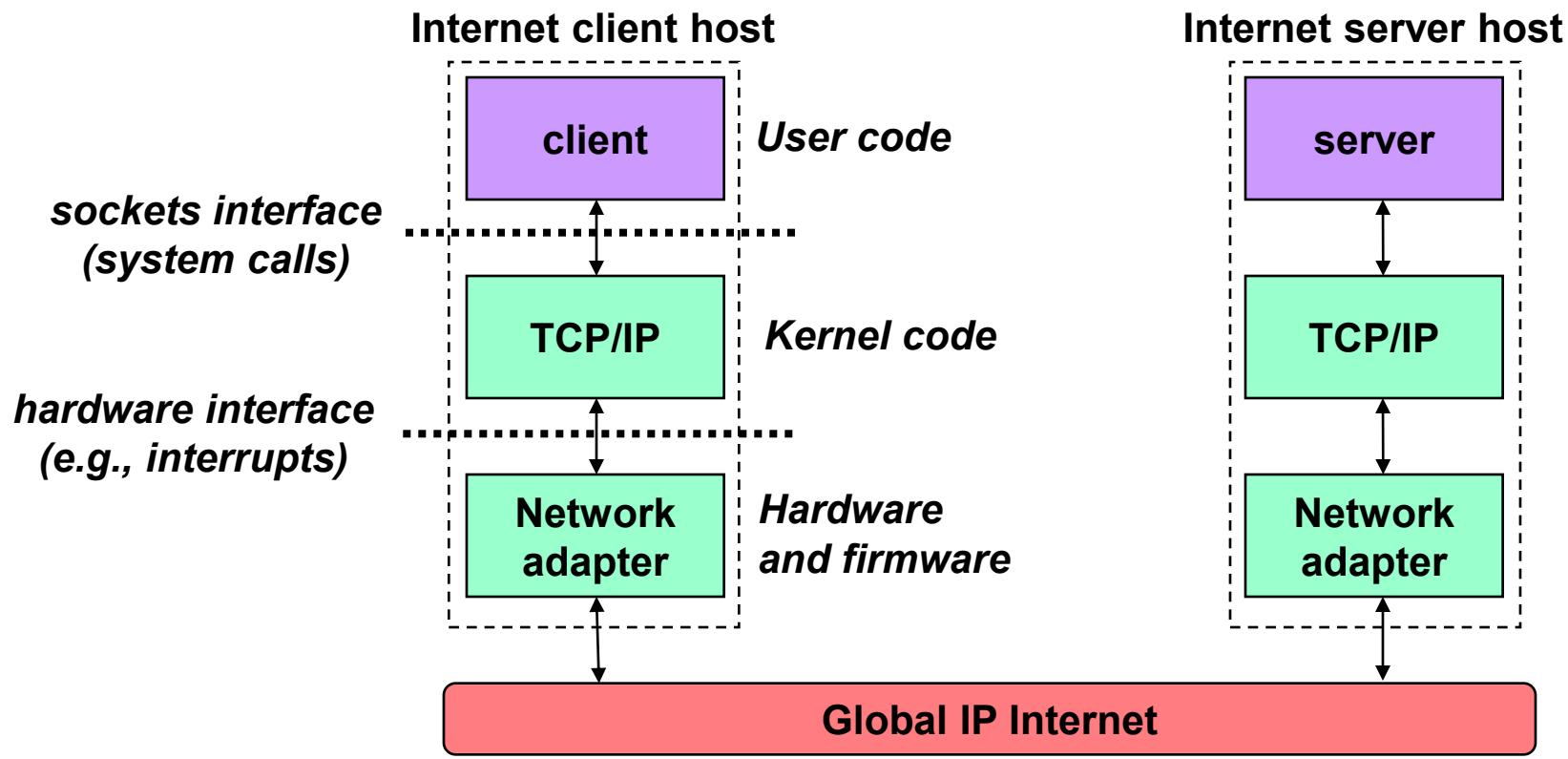


IP (Internet protocol) offers naming scheme + delivery

Global IP Internet

- Based on the TCP/IP protocol family.
  - UDP (Unreliable Datagram Protocol)
    - uses IP to provide unreliable datagram delivery from process-to-process
  - TCP (Transmission Control Protocol)
    - uses IP to provide reliable byte streams (like files) from process-to-process
- Accessed via a mix of Unix file I/O and functions from the Berkeley sockets interface

Hardware and software organization of an Internet application



Programmer's view of the Internet

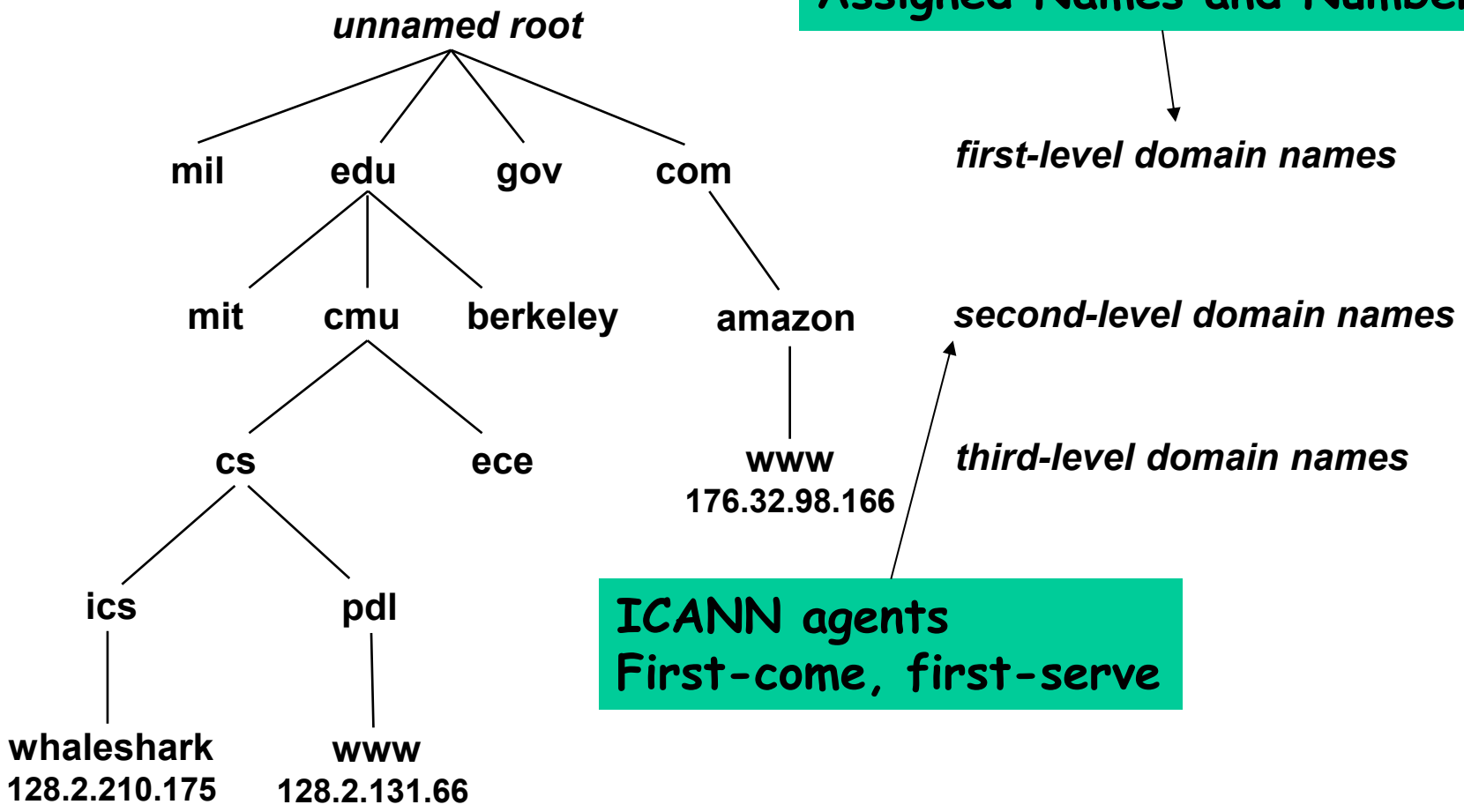
- Hosts are mapped to a set of 32-bit IP addresses
  - `0xca7828bc` (ICS server)
- The set of IP addresses is mapped to a set of identifiers called Internet domain names
  - `0xca7828bc` is mapped to ipads.se.sjtu.edu.cn
- A process on one host communicates with a process on another host over a connection

Dotted decimal notation

- By convention, each byte in a 32-bit IP address is represented by its decimal value and separated by a period
  - IP address `0xca7828bc` = `202.120.40.188`
  - IP addresses are stored in memory in network byte order (**`big-endian`** byte order)

Internet Domain Names

ICANN:
Internet Corporation for
Assigned Names and Numbers



ICANN agents
First-come, first-serve

Domain Naming System (DNS)

- The Internet maintains a mapping between IP addresses and domain names in a huge distributed database called DNS
- Conceptually, we can think of the DNS database as being millions of host entry structures
- Each host entry is an equivalence class of domain names and IP addresses

Recall "DNS服务器" in the network configuration.

Properties of DNS host entries

- **1-1:** mapping between domain name and IP address:

  linux> nslookup whaleshark.ics.cs.cmu.edu
  Address: 128.2.210.175

- **M-1:** Multiple domain names mapped to the same IP address

  linux> nslookup cs.mit.edu
  Address: 18.62.1.6

  linux> nslookup eecs.mit.edu
  Address: 18.62.1.6

Properties of DNS host entries

- **M-N:** Multiple domain names mapped to multiple IP addresses

```
linux> nslookup www.twitter.com
Address: 199.16.156.6
Address: 199.16.156.70
Address: 199.16.156.102
Address: 199.16.156.230

linux> nslookup twitter.com
Address: 199.16.156.6
Address: 199.16.156.70
Address: 199.16.156.102
Address: 199.16.156.230
```

Internet Connections for Applications

- Clients and servers communicate by sending streams of bytes of connections
  - point-to-point
    - it connects a pair of processes
  - full-duplex
    - data can flow in both directions at the same time
  - reliable
    - data sent by source process will eventually be received by the destination process in the same order it was sent

Q: How to name a process?
Currently, we can only name a host by using IP/domain name.

Socket

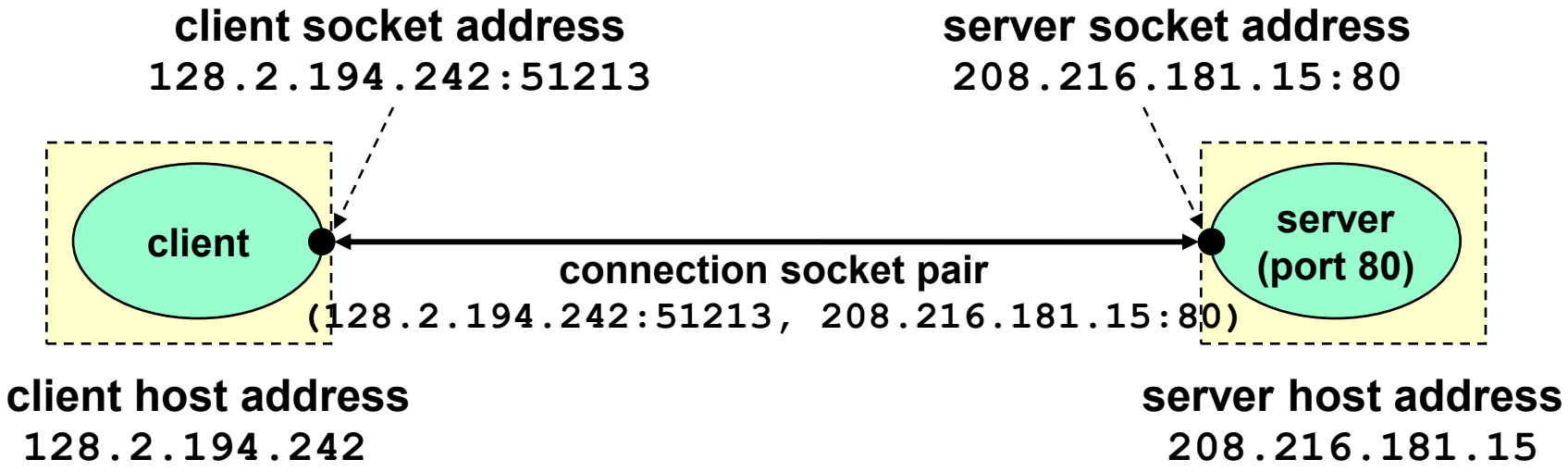
- A socket is an endpoint of a connection
  - Socket address is an `IPaddress:port` pair
- A port is a 16-bit integer that identifies a process
- Ephemeral client socket port
  - Assigned automatically on client when client makes a connection request

Socket

- Well-known server port:
  - associated with some service provided by a server
  - e.g., port 80 is associated with Web servers, port 25 is associated with email
  - with well-known name such as http (web service), smtp (email) contained in a file /etc/services

- A connection is uniquely identified by the socket addresses of its endpoints (socket pair)
  - **(cliaddr:cliport, servaddr:servport)**

Putting it all together: Anatomy of an Internet connection



Clients

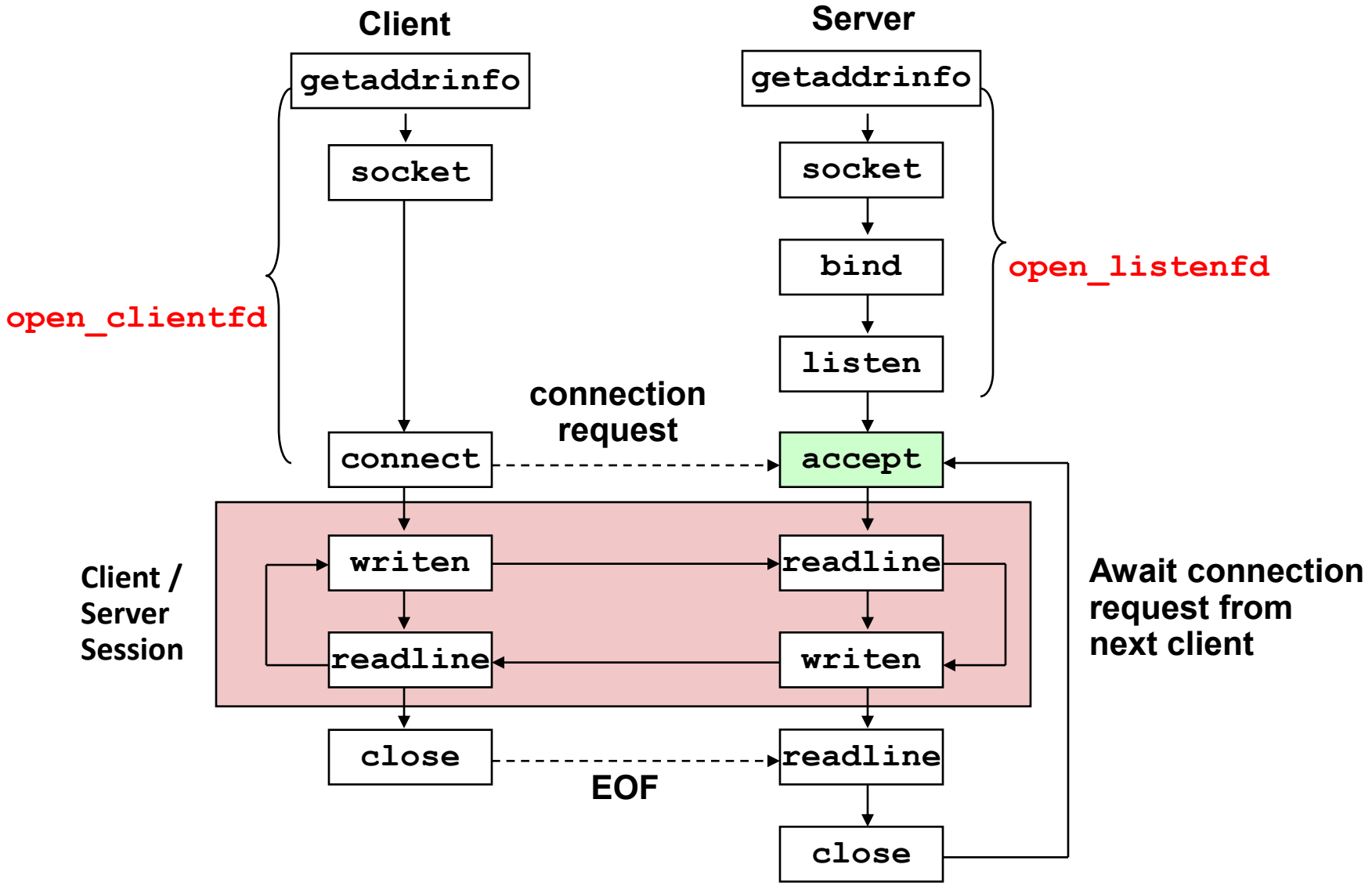
- Examples of client programs
  - **`Web browsers, ftp, telnet, ssh`**
  - **`MCP client, LLM Desktop`**
- How does the client find the server?
  - Optional: get IP-address through DNS
  - The address of the server process has two parts: **`IP-address:port`**
    - The IP address is a unique 32-bit positive integer that identifies the server host
      - dotted decimal form: `0x8002C2F2 = 128.2.194.242`
    - The port is 16-bit positive integer associated with a service (and thus a server process) on that machine

Servers

- Servers could be long-running processes (daemons)
  - Created at boot-time typically
  - Run continuously until the machine is turned off
- A machine that runs a server process is also often referred to as a "server"

SOCKETS INTERFACE

Overview of the Sockets Interface



What is a socket?

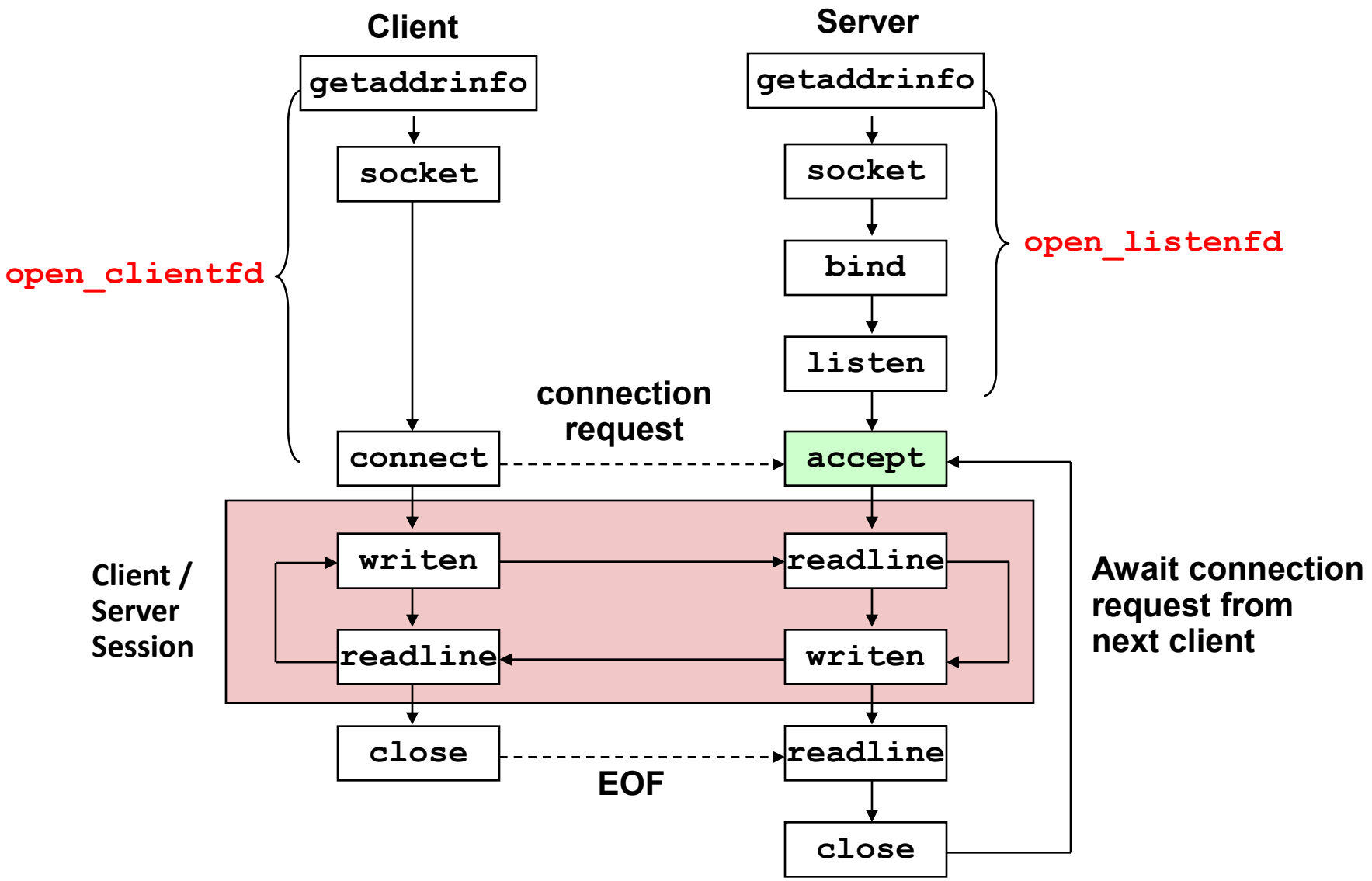
- From the perspective of a Linux program
  - a descriptor that lets an application read/write from/to the network

- Key idea:
  - Unix uses the same abstraction for both regular file I/O and network I/O

What is a socket?

- Clients and servers communicate with each other by reading from and writing to socket descriptors
  - Using regular Unix **`read`** and **`write`** I/O functions
- The main difference between file I/O and socket I/O is how the application "opens" the socket descriptors

Overview of the Sockets Interface



Echo client

```
int main(int argc, char **argv)
{
    int clientfd;
    char *host, *port, buf[MAXLINE];
    rio_t rio;

    if (argc != 3) {
       fprintf(stderr,"usage:%s <host> <port>\n",argv[0]);
       exit(0);
    }
    host = argv[1]; port = argv[2];
    clientfd = open_clientfd(host, port);
    Rio_readinitb(&rio, clientfd) ;

    while (Fgets(buf, MAXLINE, stdin) != NULL) {
       Rio_writen(clientfd, buf, strlen(buf));
       Rio_readline(&rio, buf, MAXLINE);
       Fputs(buf, stdout);
    }
    Close(clientfd);
}
```

Echo client:open_clientfd()

```
1 int open_clientfd(char *hostname, char *port) {
2         int clientfd;
3         struct addrinfo hints, *listp, *p;
4
5         /* Get a list of potential server addresses */
6         memset(&hints, 0, sizeof(struct addrinfo));
7         hints.ai_socktype = SOCK_STREAM; /* Open a connection */
8         hints.ai_flags = AI_NUMERICSERV;  /* Use a numeric port arg @port */
9         hints.ai_flags |= AI_ADDRCONFIG;   /* Recommended for connections */
10        getaddrinfo(hostname, port, &hints, &listp);
11
```

- getaddrinfo: The client prepares the server's Internet address

Echo client

```
12      /* Walk the list for one that we can successfully connect to */
13      for (p = listp; p; p = p->ai_next) {
14          /* Create a socket descriptor */
15          if ((clientfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol)) < 0)
16              continue;           /* Socket failed, try the next */
17
18          /* Connect to the server */
19          if (connect(clientfd, p->ai_addr, p->ai_addrlen) != -1)
20              break;              /* Success */
21          Close(clientfd);        /* Connect failed, try another */
22      }
23
```

Echo client: `open_clientfd()` (socket)

- The client creates a socket that will serve as the endpoint of an Internet (AF_INET) connection (SOCK_STREAM).
  - `socket()` returns an integer socket descriptor

```
int clientfd;   /* socket descriptor */

clientfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol)
/* clientfd = socket(AF_INET, SOCK_STREAM, 0);*/
```

Echo client

```
12      /* Walk the list for one that we can successfully connect to */
13      for (p = listp; p; p = p->ai_next) {
14          /* Create a socket descriptor */
15          if ((clientfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol)) < 0)
16              continue;           /* Socket failed, try the next */
17
18          /* Connect to the server */
19          if (connect(clientfd, p->ai_addr, p->ai_addrlen) != -1)
20              break;              /* Success */
21          Close(clientfd);        /* Connect failed, try another */
22      }
23
```

Echo client: `open_clientfd()` (connect)

- The client creates a connection with the server
  - The client process **suspends** (blocks) until the connection is created with the server
  - At this point the client is ready to begin exchanging messages with the server via Unix I/O calls on the descriptor **clientfd**

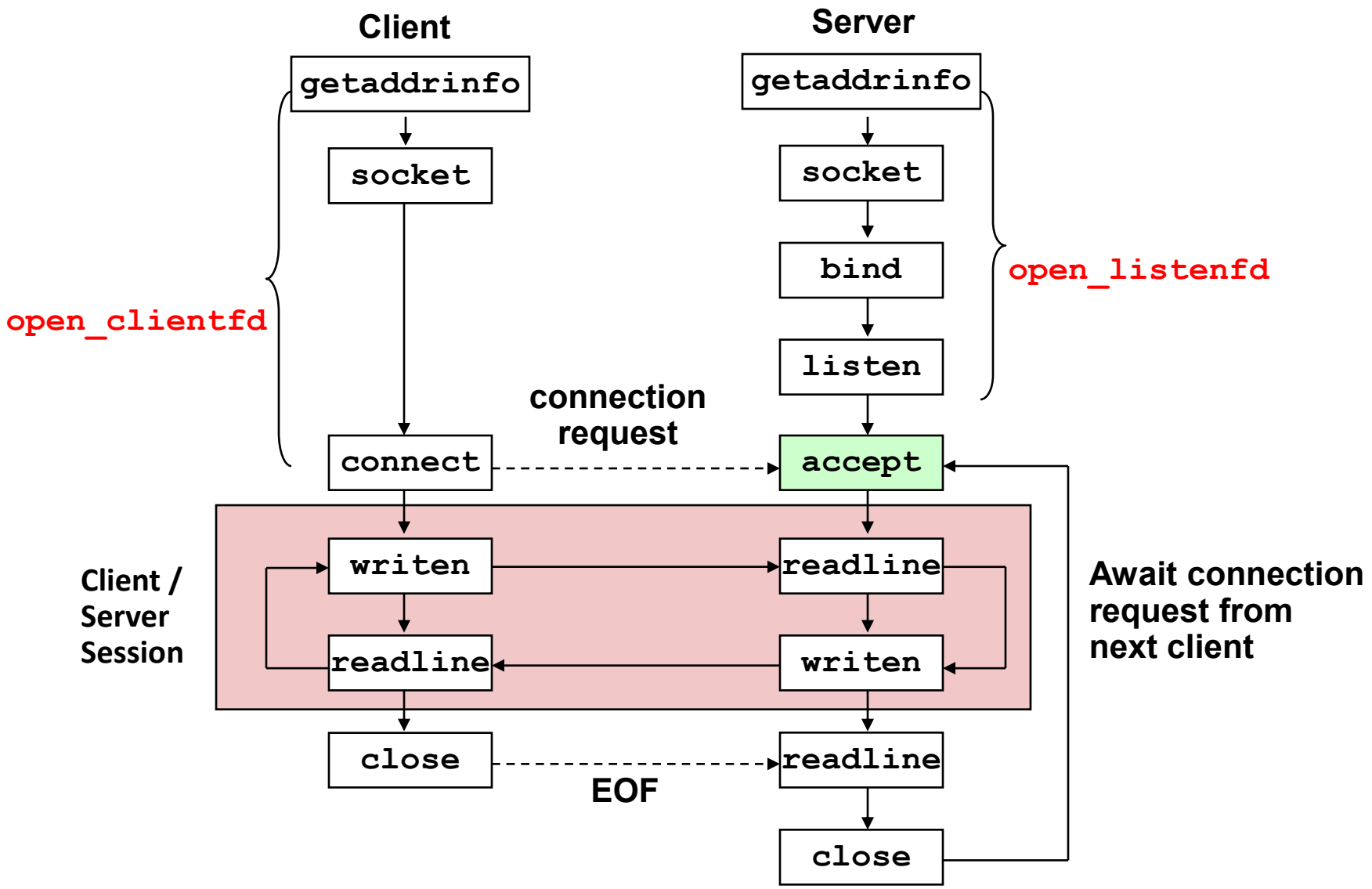
```
int clientfd;           /* socket descriptor */
struct addrinfo *p;     /* server address */

/* establish a connection with the server */
connect(clientfd, p->ai_addr, p->ai_addrlen);
```

Echo client

```
24          /* Clean up */
25          Freeaddrinfo(listp);
26          if (!p)                 /* All connects failed */
27              return -1;
28          else                    /* Succeeded */
29              return clientfd;
30  }
```

Overview of the Sockets Interface



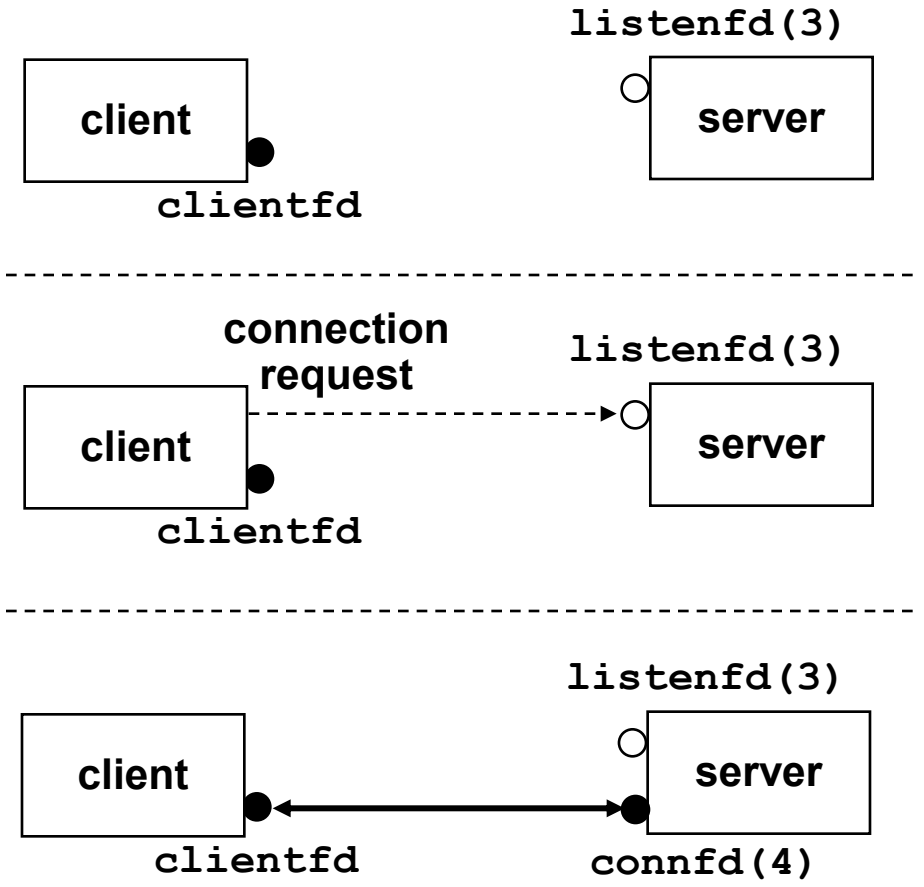
Echo server

```
1   #include "csapp.h"
2
3   void echo(int connfd);
4
5   int main(int argc, char **argv)
6   {
7           int listenfd, connfd;
8           socklen_t clientlen;
9           struct sockaddr_storage clientaddr; /* Enough space for any address */
10          char client_hostname[MAXLINE], client_port[MAXLINE];
11
12          if (argc != 2) {
13              fprintf(stderr, "usage: %s <port>\n", argv[0]);
14              exit(0);
15          }
16
```

Echo server

```
17        listenfd = Open_listenfd(argv[1]);
18        while (1) {
19            clientlen = sizeof(struct sockaddr_storage);
20            connfd = accept(listenfd, (SA *)&clientaddr, &clientlen);
21            getnameinfo((SA *) &clientaddr, clientlen, client_hostname,
22                        MAXLINE, client_port, MAXLINE, 0);
23            printf("Connected to (%s, %s)\n", client_hostname, client_port);
24            echo(connfd);
25            Close(connfd);
26        }
27        exit(0);
28    }
```

`accept()` illustrated



1. Server blocks in `accept`, waiting for connection request on listening descriptor `listenfd`.

2. Client makes connection request by calling and blocking in `connect`.

3. Server returns `connfd` from `accept`. Client returns from `connect`. Connection is now established between `clientfd` and `connfd`.

Echo server: open_listenfd()

```
1   int open_listenfd(char *port)
2   {
3           struct addrinfo hints, *listp, *p;
4           int listenfd, optval=1;
5
6           /* Get a list of potential server addresses */
7           memset(&hints, 0, sizeof(struct addrinfo));
8           hints.ai_socktype = SOCK_STREAM; /* Accept connections */
9           hints.ai_flags = AI_PASSIVE | AI_ADDRCONFIG;
10          hints.ai_flags |= AI_NUMERICSERV; /* using @port number */
11          getaddrinfo(NULL, port, &hints, &listp);
12
```

AI_PASSIVE instructs getaddrinfo() to return socket addresses that can be used by servers as listening sockets.

In this case, the host argument should be NULL.

Echo s

```
13      /* Walk the list for one that we can bind to */
14      for (p = listp; p; p = p->ai_next) {
15          /* Create a socket descriptor */
16          if ((listenfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol)) < 0)
17              continue;           /* Socket failed, try the next */
18
19          /* Eliminates "Address already in use" error from bind */
20          setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR,
21                     (const void *)&optval , sizeof(int));
22
23          /* Bind the descriptor to the address */
24          if (bind(listenfd, p->ai_addr, p->ai_addrlen) == 0)
25              break;              /* Success */
26          Close(listenfd);        /* Bind failed, try the next */
27      }
```

Echo server: `open_listenfd()` (`bind`)

- `bind()` associates the socket with the socket address we just created.

```
int listenfd;              /* listening socket */
struct addrinfo *p;        /*server's socket addr*/

/* listenfd will be an endpoint for all requests
   to port on any IP address for this host */
bind(listenfd, p->ai_addr, p->ai_addrlen);
```

Echo server: open_listenfd()

```
28
29          /* Clean up */
30          freeaddrinfo(listp);
31          if (!p)     /* No address worked */
32              return -1;
33
34          /* Make it a listening socket ready to accept connection requests */
35          if (listen(listenfd, LISTENQ) < 0) {
36              Close(listenfd);
37              return -1;
38          }
39          return listenfd;
40  }
```

Echo server: `open_listenfd (listen)`

- **`listen()`** indicates that this socket will accept connection (**`connect`**) requests from clients.

```
int listenfd;                          /* listening socket */

/* make listenfd it a server-side listening socket
ready to accept connection requests from clients */
listen(listenfd, LISTENQ);
```

- The backlog argument is a hint about the number of outstanding connection requests that the kernel should queue up before it starts to refuse requests

Echo server

We're finally ready to enter the main server loop that accepts and processes client connection requests

```
17        listenfd = Open_listenfd(argv[1]);
18        while (1) {
19            clientlen = sizeof(struct sockaddr_storage);
20            connfd = accept(listenfd, (SA *)&clientaddr, &clientlen);
21            getnameinfo((SA *) &clientaddr, clientlen, client_hostname,
22                        MAXLINE, client_port, MAXLINE, 0);
23            printf("Connected to (%s, %s)\n", client_hostname, client_port);
24            echo(connfd);
25            Close(connfd);
26        }
27        exit(0);
28    }
```

Echo server: main loop

- The server loops endlessly, waiting for connection requests, then reading input from the client, and echoing the input back to the client

```
main() {
    /* create and configure the listening socket */

    while(1) {
        /* accept(): wait for a connection request */
        /* echo(): read/echo input line from client */
        /* Close(): close the connection */
    }
}
```

Echo server: `accept()`

- `accept()` blocks waiting for a connection request

```
int listenfd; /* listening descriptor */
int connfd;   /* connected descriptor */
socketlen_t clinetlen;
struct sockaddr_storage clientaddr;


clientlen = sizeof(clientaddr);
connfd = accept(listenfd,
        (SA *)&clientaddr, &clientlen);
```

Echo server: `accept()`

- **`accept()`** returns a connected socket descriptor (**`connfd`**)
  - Returns when connection between client and server is complete
  - All I/O with the client will be done via the connected socket
- **`accept()`** also fills in client's address
  - The server can determine the domain name and IP address of the client

Echo server: `echo()`

- The server uses Unix I/O to read and echo text lines until EOF (end-of-file) is encountered
  - EOF notification caused by client calling

    `close(clientfd)`
  - NOTE: EOF is a condition, not a data byte

Echo server: echo()

```
void echo(int connfd)
{
        size_t n;
        char buf[MAXLINE];
        rio_t rio

        Rio_readinitb(&rio, connfd)
        while((n = Rio_readlineb(&rio, buf, MAXLINE))!=0){
            printf("server received %d bytes\n", n);
            Rio_writen(connfd, buf, n);
        }
}
```

Homework (Optional)

- It is strongly suggested you vibe and run the code on your own machine!

HTTP

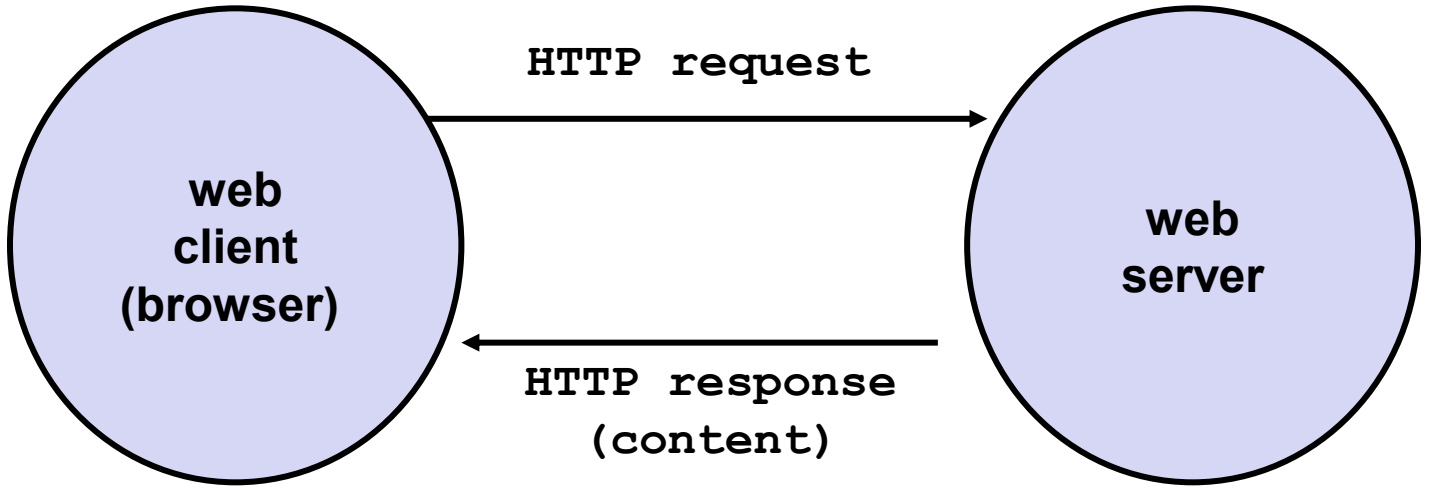
Web servers

- Clients and servers communicate using the HyperText Transfer Protocol (HTTP)
  - Client and server establish TCP connection
  - Client requests content
  - Server responds with requested content
  - Client and server close connection (usually)
- Current version is HTTP/1.1
  - RFC 2616, June, 1999.

Tim Berners Lee
1989
Turing Award 2016

`http://www.w3.org/Protocols/rfc2616/rfc2616.html`

Web servers



HTTP	Web content
TCP	Streams
IP	Datagrams

Web content

- Web servers return content to clients
  - content: a sequence of bytes with an associated MIME (Multipurpose Internet Mail Extensions) type

- Example MIME types
  - text/html HTML page
  - text/plain Unformatted text
  - application/postscript Postscript document
  - image/gif Binary image encoded in GIF format
  - image/jpg Binary image encoded in JPG format

Static and dynamic content

- The content returned in HTTP responses can be either static or dynamic
  - Static content:
    - Content stored in files and retrieved in response to an HTTP request
    - Examples: HTML files, images, audio clips
  - Dynamic content:
    - Content produced on-the-fly in response to an HTTP request
    - Example: content produced by a program executed by the server on behalf of the client

URLs

- Each file managed by a server has a unique name called a URL (Universal Resource Locator)

- URLs for static content:
  - identifies a file like `index.html`, managed by a Web server at `ipads.se.sjtu.edu.cn` that is listening on port 80

http://ipads.se.sjtu.edu.cn:80/courses/ics/index.shtml
http://ipads.se.sjtu.edu.cn:/courses/ics/index.shtml
http://ipads.se.sjtu.edu.cn/courses/ics/

URLs

- URLs for dynamic content:
  - Identifies an executable file called **adder**, managed by a Web server at `www.cs.cmu.edu` that is listening on port 8000, that should be called with two argument strings: `15000` and `213`

http://www.cs.cmu.edu:8000/cgi-bin/adder?15000&213

How clients and servers use URLs

- Example URL:

  `http://www.aol.com:80/index.html`

- Clients use prefix (`http://www.aol.com:80`) to infer:
  - What kind of server to contact (Web server)
  - Where the server is (www.aol.com)
  - What port it is listening on (80)

How clients and servers use URLs

- Servers use suffix (`/index.html`) to:
  - Determine if request is for static or dynamic content
    - No hard and fast rules for this
    - Convention: executables reside in cgi-bin directory
  - Find file on filesystem
    - Initial "/" in suffix denotes home directory for requested content
    - Minimal suffix is "/", which all servers expand to some default home page (e.g., index.html)

Anatomy of an HTTP transaction

```
//Client: open connection to server
unix> telnet ipads.se.sjtu.edu.cn 80
//Telnet prints 3 lines to the terminal
Trying 202.120.40.85...
Connected to ipads.se.sjtu.edu.cn.
Escape character is '^]'.
//Client: request line
GET /courses/ics/index.shtml HTTP/1.1
//Client: required HTTP/1.1 HOST header
host: ipads.se.sjtu.edu.cn
//Client: empty line terminates headers
```

HTTP Requests

- HTTP request is a request line, followed by zero or more request headers
  - empty line terminates headers

- Request line: <method> <uri> <version>
  - <version> is HTTP version of request (`HTTP/1.0` or `HTTP/1.1`)
  - <uri> is typically URL for proxies, URL suffix for servers.
    - A URL is a type of URI (Uniform Resource Identifier)
  - <method> is either `GET`, `POST`, `OPTIONS`, `HEAD`, `PUT`, `DELETE`, or `TRACE`

HTTP Requests (cont)

- HTTP methods:
  - **GET**: Retrieves static or dynamic content
    - Arguments for dynamic content are in URI
    - Workhorse method (99% of requests)
  - **POST**: Requests server to accept the data enclosed in the body of the request message. For example,
    - Retrieve dynamic content
    - Arguments for dynamic content are in the request body

- Request headers: **<header name>: <header data>**
  - Provide additional information to the server
  - e.g., `host: ipads.se.sjtu.edu.cn`

Anatomy of an HTTP transaction

```
//Server: response line
HTTP/1.1 200 OK
//Server: followed by five response headers
Server: nginx/1.0.4
Date: Thu, 29 Nov 2012 10:15:38 GMT
//Server: expect HTML in the response body
Content-Type: text/html
//Server: expect 11,560 bytes in the response body
Content-Length: 11560
//Server: empty line ("\r\n") terminates hdrs
//Server: first HTML line in response body
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01
Transitional//EN">
<html> ...
```

HTTP Responses

- HTTP response is a response line followed by zero or more response headers
- Response line: **<version> <status code> <status msg>**
  - **<version>** is HTTP version of the response
  - **<status code>** is numeric status
  - **<status msg>** is corresponding English text

200 OK	Request was handled without error
301 Moved	Provide alternate URL
403 Forbidden	Server lacks permission to access file
404 Not found	Server couldn't find the file
501 Not Implemented	Server does not support the request method
505 HTTP version not supported	Server does not support version in request

- Response headers: **<header name>: <header data>**
  - Provide additional information about response
  - **Content-Type**: MIME type of content in response body
  - **Content-Length**: Length of content in response body